All these examples look simple enough: define an event and then call (emit or trigger) it. Now let's see how you use these concepts and access a triggered event to notify users of what's happening in your application. We'll reuse the blind auction use case from chapter 5, but with the significant addition of events and related modifications to the code.

6.2.2 Blind auction with events

Online auctions and decentralized marketplaces are ideal use cases for blockchain. The blind auction introduced in chapter 5 has four phases: `Init`, `Bidding`, `Reveal`, and `Done` (auction ended). Wouldn't it be nice if you could notify users at the beginning of the `Bidding` and `Reveal` phases so that they were ready to act (bid or reveal) and didn't miss a deadline during any of the blind auction phases? In this chapter, you'll add three events: `AuctionEnded`, `BiddingStarted`, and `RevealStarted`.

Why are we discussing events in the context of on-chain data? As shown in figure 6.2, events are logged in the block's receipt tree, so events are good examples of on-chain data. Event logs can be used for near-real-time responses, as in the blind auction use case, and also for offline indexing, querying, searching, and analytics of on-chain data. Events (when triggered) create logs that can be indexed and searched by topic. The topics in this case are the event name itself and the individual parameters. This type of fine-grained access is useful for the analysis of a blockchain's historical data.

There are many ways to access emitted events, including using listeners (the push method) and using receipt logs (the pull method). In this example, we'll use the latter approach to